07-SYSTEM-DIAGRAMS.md

.things that Matter

System Analysis & UML Diagrams

Document Information	
Company Name	.things that Matter
Document Type	System Analysis & Diagrams
Version	1.0
Date	January 21, 2026
Author	Matome Mboweni, CTO

Table of Contents

1. \1 (\2)
2. \1 (\2)
3. \1 (\2)
4. \1 (\2)
5. \1 (\2)
6. \1 (\2)
7. \1 (\2)
8. \1 (\2)
9. \1 (\2)
10. \1 (\2)

1. C4 Model Diagrams

1.1 System Context Diagram (Level 1)

Shows the system's boundaries and interactions with external actors.

```
C4Context
    title System Context Diagram - .things that Matter

    Person(merchant, "Merchant", "E-commerce business owner who wants to distribute products b
eyond their storefront")
    Person(consumer, "Consumer", "Online shopper who discovers and purchases products")
    Person(developer, "Developer", "Third-party integrator building Matter-enabled apps")
    Person(aiAgent, "AI Agent", "Autonomous commerce bot that processes product data efficient
ly")

    System(matterPlatform, ".things that Matter Platform", "Commerce infrastructure that trans
forms products into portable, self-sovereign digital objects")

    System_Ext(socialMedia, "Social Media Platforms", "Discord, Telegram, Twitter, etc.")
    System_Ext(clipApi, "CLIP API (Replicate)", "AI model for semantic vectorization")
    System_Ext(cloudflareR2, "Cloudflare R2", "Object storage CDN")
    System_Ext(merchantStore, "Merchant Storefront", "Original e-commerce platform")
    System_Ext(chromeStore, "Chrome Web Store", "Browser extension distribution")
```

```
    Rel(merchant, matterPlatform, "Uploads product catalogs", "HTTPS/JSON")
    Rel(consumer, matterPlatform, "Discovers products via extension", "Browser Extension")
    Rel(developer, matterPlatform, "Integrates via SDK/API", "REST API")
    Rel(aiAgent, matterPlatform, "Reads .thing files efficiently", "Binary Protocol")

    Rel(matterPlatform, clipApi, "Generates semantic vectors", "HTTPS")
    Rel(matterPlatform, cloudflareR2, "Stores artifacts", "S3 API")
    Rel(matterPlatform, socialMedia, "Distributes polyglot images", "Image Upload")
    Rel(consumer, merchantStore, "Completes purchase", "Redirect")
    Rel(matterPlatform, chromeStore, "Publishes extension", "Store API")
```

\1

```
                            EXTERNAL ACTORS


    Merchant         Consumer         AI Agent         Developer
   (Product Owner)    (Shopper)          (Bot)              (Integrator)

         Upload           Discover          Parse              Integrate
         Catalog          Products          .thing             SDK



                    .things that Matter Platform

      "Commerce infrastructure for portable, self-sovereign products"




    CLIP          Cloudflare        Social           Chrome
    API              R2             Media          Web Store
   (Replicate)      (CDN)          Platforms
```

1.2 Container Diagram (Level 2)

Shows the high-level technical building blocks.

```
C4Container
    title Container Diagram - .things that Matter Platform

    Person(merchant, "Merchant", "Uploads products")
    Person(consumer, "Consumer", "Discovers products")

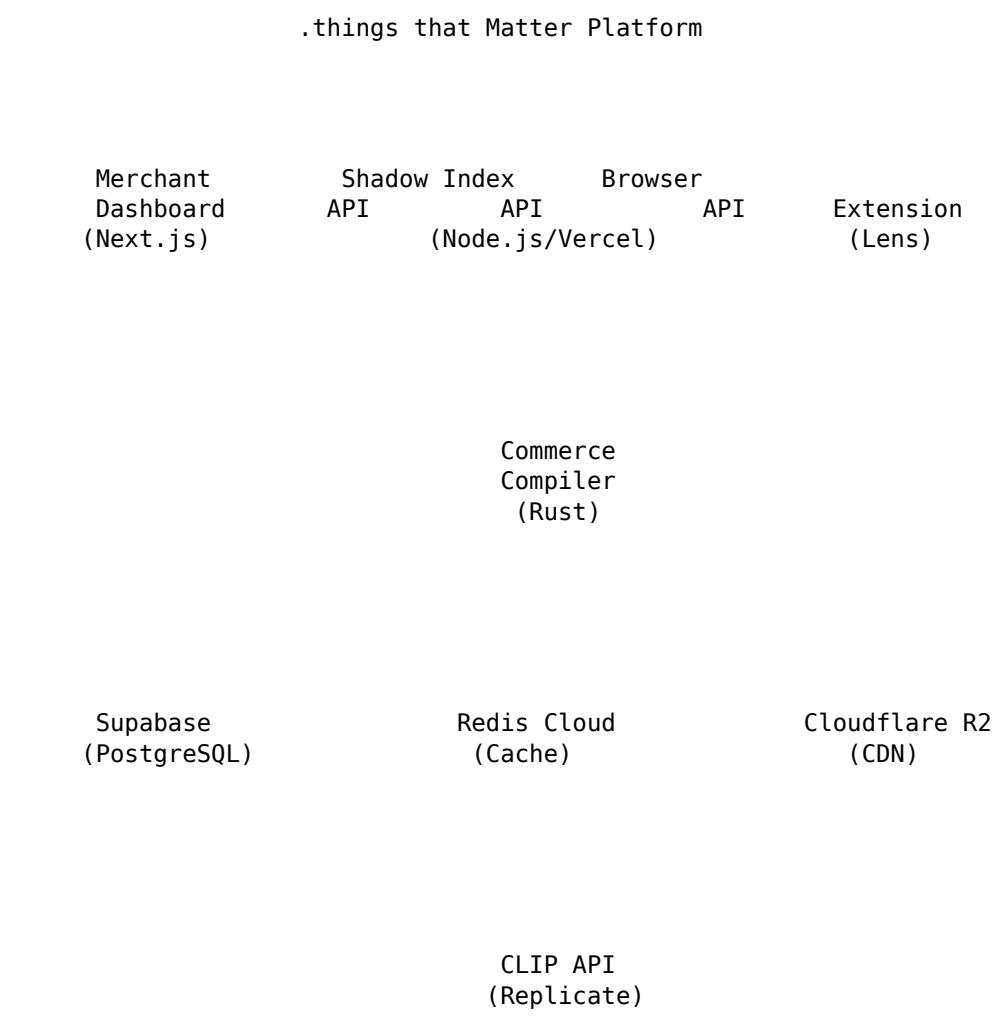
    System_Boundary(platform, ".things that Matter Platform") {
        Container(compiler, "Commerce Compiler", "Rust", "Transforms product data into .thing
files")
        Container(shadowIndex, "Shadow Index API", "Node.js/Vercel", "Hash-to-artifact resolut
ion service")
        Container(extension, "Browser Extension (Lens)", "JavaScript/WASM", "Detects and displ
ays Matter products")
        Container(dashboard, "Merchant Dashboard", "Next.js", "Self-service product management
")
        ContainerDb(supabase, "Database", "PostgreSQL/Supabase", "Stores hash mappings and met
adata")
```

```
        ContainerDb(redis, "Cache", "Redis Cloud", "Hot lookup caching")
    }

    System_Ext(clipApi, "CLIP API", "Semantic vectorization")
    System_Ext(r2, "Cloudflare R2", "Artifact storage")

    Rel(merchant, dashboard, "Manages products", "HTTPS")
    Rel(dashboard, shadowIndex, "Triggers compilation", "REST API")
    Rel(shadowIndex, compiler, "Invokes", "Function call")
    Rel(compiler, clipApi, "Vectorizes images", "HTTPS")
    Rel(compiler, r2, "Stores artifacts", "S3 API")
    Rel(shadowIndex, supabase, "Reads/Writes", "PostgreSQL")
    Rel(shadowIndex, redis, "Caches lookups", "Redis Protocol")
    Rel(consumer, extension, "Browses web", "Browser")
    Rel(extension, shadowIndex, "Resolves hashes", "HTTPS")
    Rel(extension, r2, "Fetches .thing files", "HTTPS")
```

\1



2. Use Case Diagrams

2.1 Merchant Use Cases

```
graph TB
```

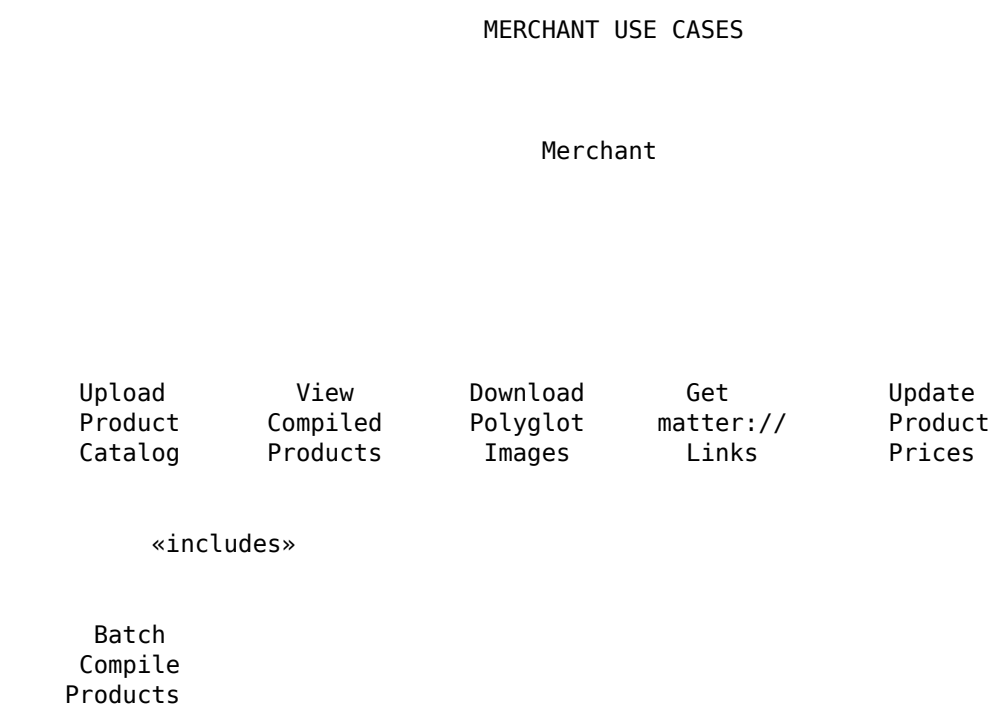
```
    subgraph "Merchant Use Cases"
        M((Merchant))

        UC1[Upload Product Catalog]
        UC2[View Compiled Products]
        UC3[Download Polyglot Images]
        UC4[Get matter:// Links]
        UC5[Update Product Prices]
        UC6[View Analytics]
        UC7[Manage API Keys]
        UC8[Batch Compile Products]

        M --> UC1
        M --> UC2
        M --> UC3
        M --> UC4
        M --> UC5
        M --> UC6
        M --> UC7
        M --> UC8

        UC1 -.->|includes| UC8
        UC5 -.->|triggers| UC1
    end
```

\1



2.2 Consumer Use Cases

```
graph TB
    subgraph "Consumer Use Cases"
        C((Consumer))

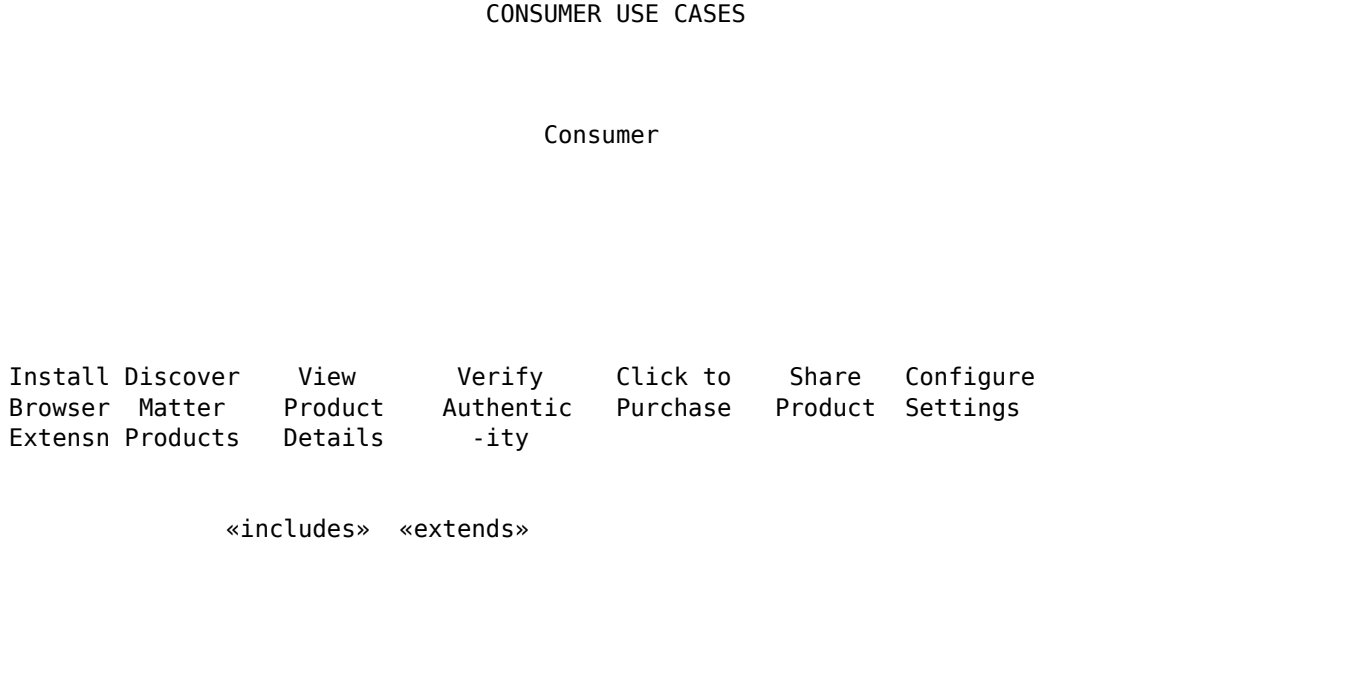
        UC1[Install Browser Extension]
        UC2[Discover Matter Products]
```

```
        UC3[View Product Details]
        UC4[Verify Product Authenticity]
        UC5[Click to Purchase]
        UC6[Share Product with Friends]
        UC7[Configure Extension Settings]
        UC8[View Purchase History]

        C --> UC1
        C --> UC2
        C --> UC3
        C --> UC4
        C --> UC5
        C --> UC6
        C --> UC7
        C --> UC8

        UC2 -.->|includes| UC4
        UC3 -.->|extends| UC5
    end
```

\1



2.3 Developer Use Cases

```
graph TB
    subgraph "Developer Use Cases"
        D((Developer))

        UC1[Access Compilation API]
        UC2[Read .thing Files]
        UC3[Verify Signatures]
        UC4[Perform Semantic Search]
        UC5[Batch Process Products]
        UC6[Integrate SDK]
        UC7[View API Documentation]
        UC8[Monitor API Usage]

        D --> UC1
        D --> UC2
        D --> UC3
```

```
        D --> UC4
        D --> UC5
        D --> UC6
        D --> UC7
        D --> UC8

        UC1 -.->|includes| UC3
        UC2 -.->|includes| UC3
    end
```

2.4 AI Agent Use Cases

```
graph TB
    subgraph "AI Agent Use Cases"
        A((AI Agent))

        UC1[Parse .thing Files O1]
        UC2[Read Product Fields Directly]
        UC3[Perform Semantic Similarity Search]
        UC4[Verify Product Authenticity]
        UC5[Batch Fetch Catalogs]
        UC6[Estimate Token Usage]
        UC7[Cache Product Data]

        A --> UC1
        A --> UC2
        A --> UC3
        A --> UC4
        A --> UC5
        A --> UC6
        A --> UC7

        UC1 -.->|enables| UC2
        UC5 -.->|includes| UC4
    end
```

3. Class Diagrams

3.1 Domain Model - Core Entities

```
classDiagram
    class Product {
        +string id
        +byte[] hash
        +uint8 version
        +string title
        +string description
        +uint64 price
        +string currency
        +string sku
        +string imageUrl
        +byte[] thumbnail
        +float[] clipVector
        +string[] categories
        +string[] tags
        +string checkoutUrl
        +string merchantId
```

```
    +InventoryStatus inventoryHint
    +uint64 createdAt
    +uint64 expiresAt
    +string sourceUrl
    +byte[] signature
    +byte[] signerPubkey
    +validate() bool
    +toFlatBuffer() byte[]
    +fromFlatBuffer(byte[]) Product
}

class ThingFile {
    +uint32 magic
    +uint8 version
    +Product product
    +Gist gist
    +serialize() byte[]
    +deserialize(byte[]) ThingFile
    +verify() bool
}

class Gist {
    +string title
    +uint64 price
    +string currency
    +string imageUrl
    +toJSON() string
}

class InventoryStatus {
    <<enumeration>>
    UNKNOWN
    IN_STOCK
    LOW_STOCK
    OUT_OF_STOCK
}

class PolyglotImage {
    +byte[] imageData
    +XMPMetadata xmpBeacon
    +byte[] eofPayload
    +string mimeType
    +embed(ThingFile) PolyglotImage
    +extract() ThingFile
}

class XMPMetadata {
    +string hash
    +string gist
    +uint8 version
    +toXML() string
    +fromXML(string) XMPMetadata
}

class MatterURI {
    +string hash
    +map~string,string~ params
    +toString() string
    +parse(string) MatterURI
}

ThingFile "1" *-- "1" Product
ThingFile "1" *-- "1" Gist
Product "1" --> "1" InventoryStatus
PolyglotImage "1" *-- "1" XMPMetadata
PolyglotImage "1" o-- "1" ThingFile
```

```
    MatterURI "1" --> "1" ThingFile
```

\1

```
                           DOMAIN MODEL


       ThingFile                           PolyglotImage

 +magic: uint32                        +imageData: byte[]
 +version: uint8                       +xmpBeacon: XMPMetadata
 +product: Product           +eofPayload: byte[]
 +gist: Gist                           +mimeType: string

 +serialize(): byte[]                  +embed(): PolyglotImage
 +deserialize(): ThingFile             +extract(): ThingFile
 +verify(): bool



        Product                              XMPMetadata

 +id: string                           +hash: string
 +hash: byte[]                         +gist: string
 +title: string                        +version: uint8
 +description: string
 +price: uint64                        +toXML(): string
 +currency: string                     +fromXML(): XMPMetadata
 +clipVector: float[512]
 +checkoutUrl: string
 +signature: byte[]
 +signerPubkey: byte[]                       MatterURI

 +validate(): bool                     +hash: string
 +toFlatBuffer(): byte[]               +params: map

                                        +toString(): string
                                        +parse(): MatterURI

   <<enumeration>>
   InventoryStatus
                  Gist
 UNKNOWN
 IN_STOCK                              +title: string
 LOW_STOCK                             +price: uint64
 OUT_OF_STOCK                          +currency: string
          +imageUrl: string

                                         +toJSON(): string
```

3.2 Commerce Compiler Architecture

```
classDiagram
    class CommerceCompiler {
        -PrivateKey signingKey
        -CLIPClient clipClient
        -Compressor compressor
```

```
    -Serializer serializer
    +compile(ProductInput) CompileResult
    +sign(byte[]) Signature
    +verify(byte[], Signature) bool
    +vectorize(Image) float[]
    +embed(ThingFile, Image) PolyglotImage
}

class ProductInput {
    +string id
    +string title
    +string description
    +uint64 price
    +string currency
    +string imageUrl
    +string checkoutUrl
    +validate() ValidationResult
}

class CompileResult {
    +string hash
    +string thingUrl
    +string imageUrl
    +string matterUri
    +Gist gist
}

class CLIPClient {
    -string apiKey
    -string endpoint
    +vectorize(Image) float[]
    +vectorizeText(string) float[]
}

class Compressor {
    +compress(byte[]) byte[]
    +decompress(byte[]) byte[]
}

class Serializer {
    +serialize(Product) byte[]
    +deserialize(byte[]) Product
}

class CryptoModule {
    +generateKeyPair() KeyPair
    +sign(byte[], PrivateKey) Signature
    +verify(byte[], Signature, PublicKey) bool
    +hash(byte[]) byte[]
}

class SteganographyEngine {
    +injectXMP(Image, XMPMetadata) Image
    +appendEOF(Image, byte[]) Image
    +extractXMP(Image) XMPMetadata
    +extractEOF(Image) byte[]
}

CommerceCompiler --> CLIPClient
CommerceCompiler --> Compressor
CommerceCompiler --> Serializer
CommerceCompiler --> CryptoModule
CommerceCompiler --> SteganographyEngine
CommerceCompiler ..> ProductInput : uses
CommerceCompiler ..> CompileResult : produces
```

3.3 Browser Extension Architecture

```
classDiagram
    class LensExtension {
        +init()
        +destroy()
    }

    class ContentScript {
        -MutationObserver observer
        +startObserving()
        +stopObserving()
        +detectBeacons(Element[])
        +injectOverlay(Product)
    }

    class BackgroundWorker {
        -Cache cache
        +fetchThingFile(string) ThingFile
        +resolveHash(string) ArtifactUrls
        +verifySignature(ThingFile) bool
    }

    class OverlayUI {
        -ShadowRoot shadowRoot
        +render(Product)
        +show()
        +hide()
        +onBuyClick()
    }

    class WASMVerifier {
        +verify(byte[], byte[], byte[]) bool
        +hash(byte[]) byte[]
    }

    class IndexedDBCache {
        +get(string) ThingFile
        +set(string, ThingFile)
        +clear()
        +getExpired() string[]
    }

    class XMPDetector {
        +detect(Image) XMPMetadata
        +hasBeacon(Image) bool
    }

    class URIHandler {
        +register()
        +handle(string)
        +parse(string) MatterURI
    }

    LensExtension --> ContentScript
    LensExtension --> BackgroundWorker
    LensExtension --> OverlayUI
    ContentScript --> XMPDetector
    ContentScript --> URIHandler
    BackgroundWorker --> WASMVerifier
    BackgroundWorker --> IndexedDBCache
    OverlayUI --> BackgroundWorker
```

4. Sequence Diagrams

4.1 Product Compilation Flow

```
sequenceDiagram
    participant M as Merchant
    participant D as Dashboard
    participant API as Shadow Index API
    participant C as Commerce Compiler
    participant CLIP as CLIP API
    participant R2 as Cloudflare R2
    participant DB as Supabase

    M->>D: Upload product JSON
    D->>API: POST /v1/compile
    API->>C: compile(productInput)

    C->>C: Validate input schema
    C->>CLIP: POST /vectorize (image)
    CLIP-->>C: 512-dim vector

    C->>C: Compress description (Zstd)
    C->>C: Serialize to FlatBuffers
    C->>C: Calculate SHA-256 hash
    C->>C: Sign with Ed25519

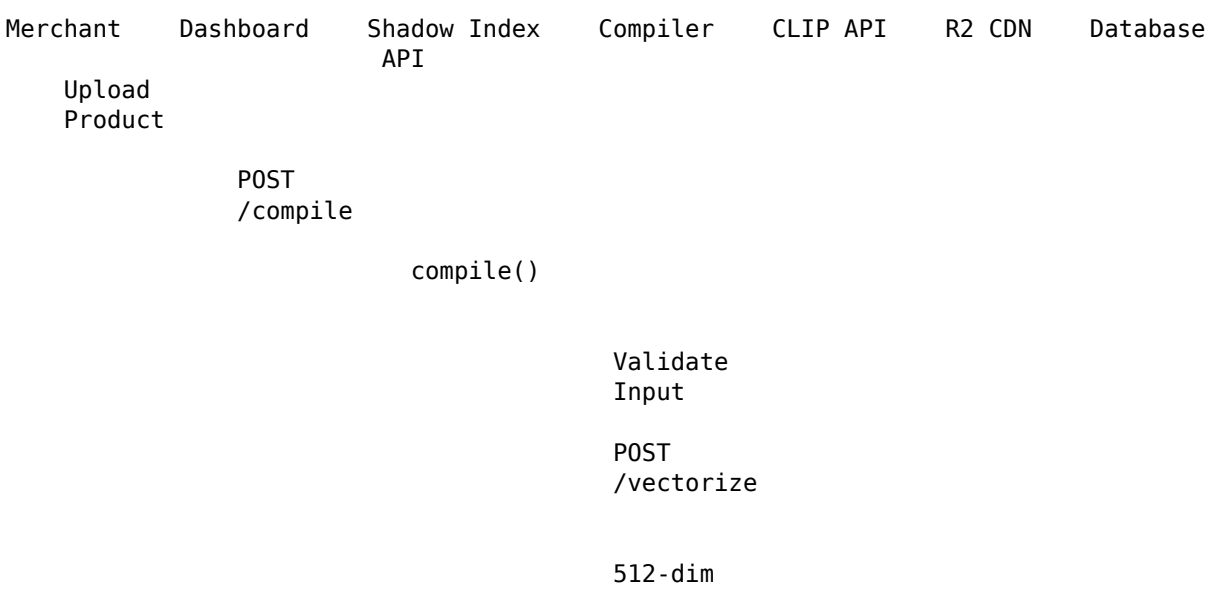
    C->>C: Generate polyglot image
    Note over C: Inject XMP beacon
    Note over C: Append EOF binary

    C->>R2: Upload .thing file
    R2-->>C: thing_url
    C->>R2: Upload polyglot image
    R2-->>C: image_url

    C-->>API: CompileResult
    API->>DB: INSERT hash mapping
    DB-->>API: OK

    API-->>D: { hash, thing_url, image_url, matter_uri }
    D-->>M: Display compiled product
```

\1



Compress,
Serialize,
Hash,
Sign

Generate
Polyglot

Upload
.thing

Upload
image

INSERT

4.2 Product Discovery Flow (Consumer)

```
sequenceDiagram
    participant C as Consumer
    participant B as Browser
    participant CS as Content Script
    participant BW as Background Worker
    participant API as Shadow Index API
    participant R2 as Cloudflare R2
    participant Cache as IndexedDB
    participant Store as Merchant Store

    C->>B: Browse social media
    B->>CS: DOM mutation (new images)
    CS->>CS: Scan for XMP beacons

    alt XMP Beacon Found
        CS->>CS: Extract hash from beacon
        CS->>BW: resolveProduct(hash)

        BW->>Cache: get(hash)
        alt Cache Hit
            Cache-->>BW: Cached ThingFile
        else Cache Miss
            BW->>API: GET /resolve/{hash}
            API-->>BW: { thing_url, gist }
            BW->>R2: GET thing_url
            R2-->>BW: .thing file bytes
            BW->>BW: Verify Ed25519 signature (WASM)
            BW->>Cache: set(hash, thingFile)
        end
```

```
        BW-->>CS: Product data
        CS->>CS: Inject Shadow DOM overlay
        CS-->>C: Display product card

        C->>CS: Click "Buy Now"
        CS->>B: Navigate to checkout_url
        B->>Store: Load checkout page
    end
```

\1

```
Consumer     Browser     Content      Background    Shadow       R2        IndexedDB    Mercha
nt
                         Script       Worker        Index API    CDN       Cache        Store

    Browse
    Page

                DOM
                Mutation


                           Scan for
                           XMP Beacons

                           Extract
                           Hash

                           resolve
                           Product


                                        Check Cache


                                        [Cache Miss]
                                        GET /resolve



                                        GET .thing



                                        Verify
                                        Signature
                                        (WASM)

                                        Cache




                           Inject
                           Overlay



    Click
    Buy Now


                Navigate
```

4.3 AI Agent Efficient Read Flow

```
sequenceDiagram
    participant Agent as AI Agent
    participant SDK as Matter SDK
    participant R2 as Cloudflare R2
    participant Local as Local Memory

    Agent->>SDK: fetchProduct(hash)
    SDK->>R2: GET /{hash}.thing
    R2-->>SDK: .thing bytes (memory-mapped)

    SDK->>SDK: Verify signature
    SDK-->>Agent: ThingFile (zero-copy)

    Agent->>SDK: readPrice()
    Note over SDK: Direct byte offset access
    Note over SDK: No parsing required
    SDK-->>Agent: price (O(1) read)

    Agent->>SDK: readClipVector()
    Note over SDK: Direct memory access
    SDK-->>Agent: float[512]

    Agent->>Local: Compare with query vector
    Local-->>Agent: Similarity score

    Note over Agent: Total: ~5ms vs ~500ms for HTML scraping
```

4.4 Batch Compilation Flow

```
sequenceDiagram
    participant M as Merchant
    participant API as Shadow Index API
    participant Q as Job Queue
    participant W as Worker
    participant C as Compiler
    participant CLIP as CLIP API
    participant R2 as R2 CDN

    M->>API: POST /v1/batch (1000 products)
    API->>API: Validate batch input
    API->>Q: Enqueue compilation jobs
    API-->>M: { batch_id, status: "processing" }

    loop For each product
        Q->>W: Dequeue job
        W->>C: compile(product)
        C->>CLIP: vectorize(image)
        CLIP-->>C: vector
        C->>C: Process & sign
        C->>R2: Upload artifacts
        W->>Q: Mark complete
    end

    M->>API: GET /v1/batch/{batch_id}
    API-->>M: { status: "complete", results: [...] }
```

5. Activity Diagrams

5.1 Product Compilation Pipeline

```
flowchart TD
    Start([Start]) --> Validate{Validate Input}
    Validate -->|Invalid| Error1[Return Validation Error]
    Error1 --> End1([End])

    Validate -->|Valid| FetchImage[Fetch Product Image]
    FetchImage -->|Failed| Error2[Return Image Fetch Error]
    Error2 --> End2([End])

    FetchImage -->|Success| Vectorize[Send to CLIP API]
    Vectorize -->|Failed| Error3[Return Vectorization Error]
    Error3 --> End3([End])

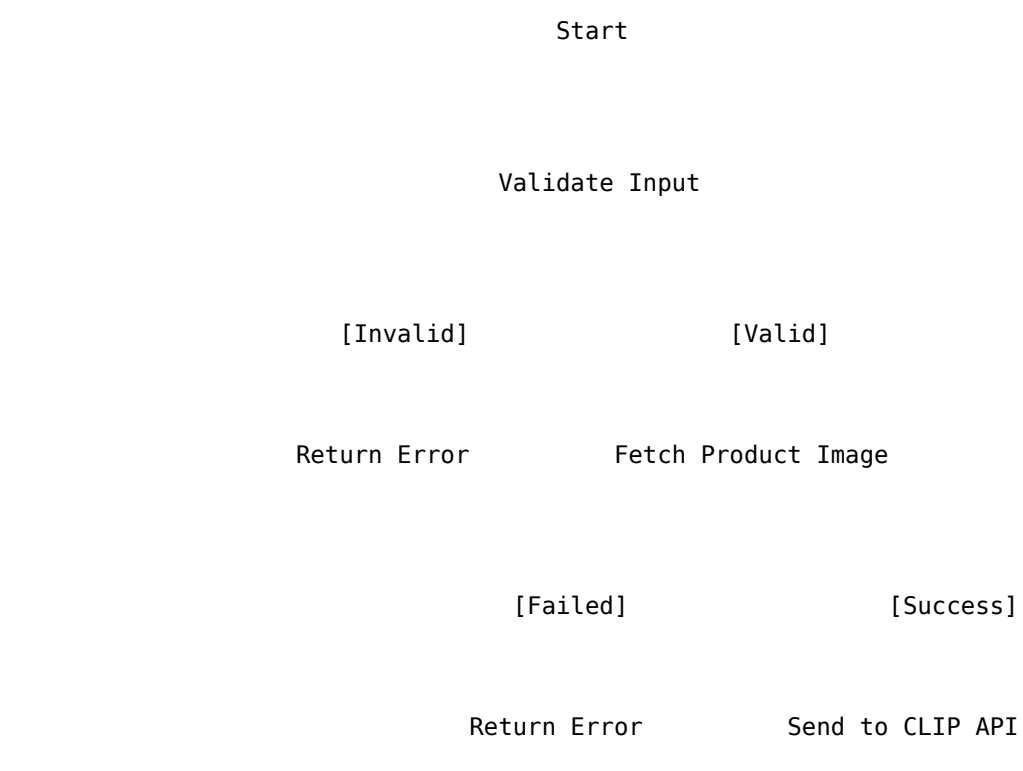
    Vectorize -->|Success| Compress[Compress Description with Zstd]
    Compress --> Serialize[Serialize to FlatBuffers]
    Serialize --> Hash[Calculate SHA-256 Hash]
    Hash --> Sign[Sign with Ed25519]

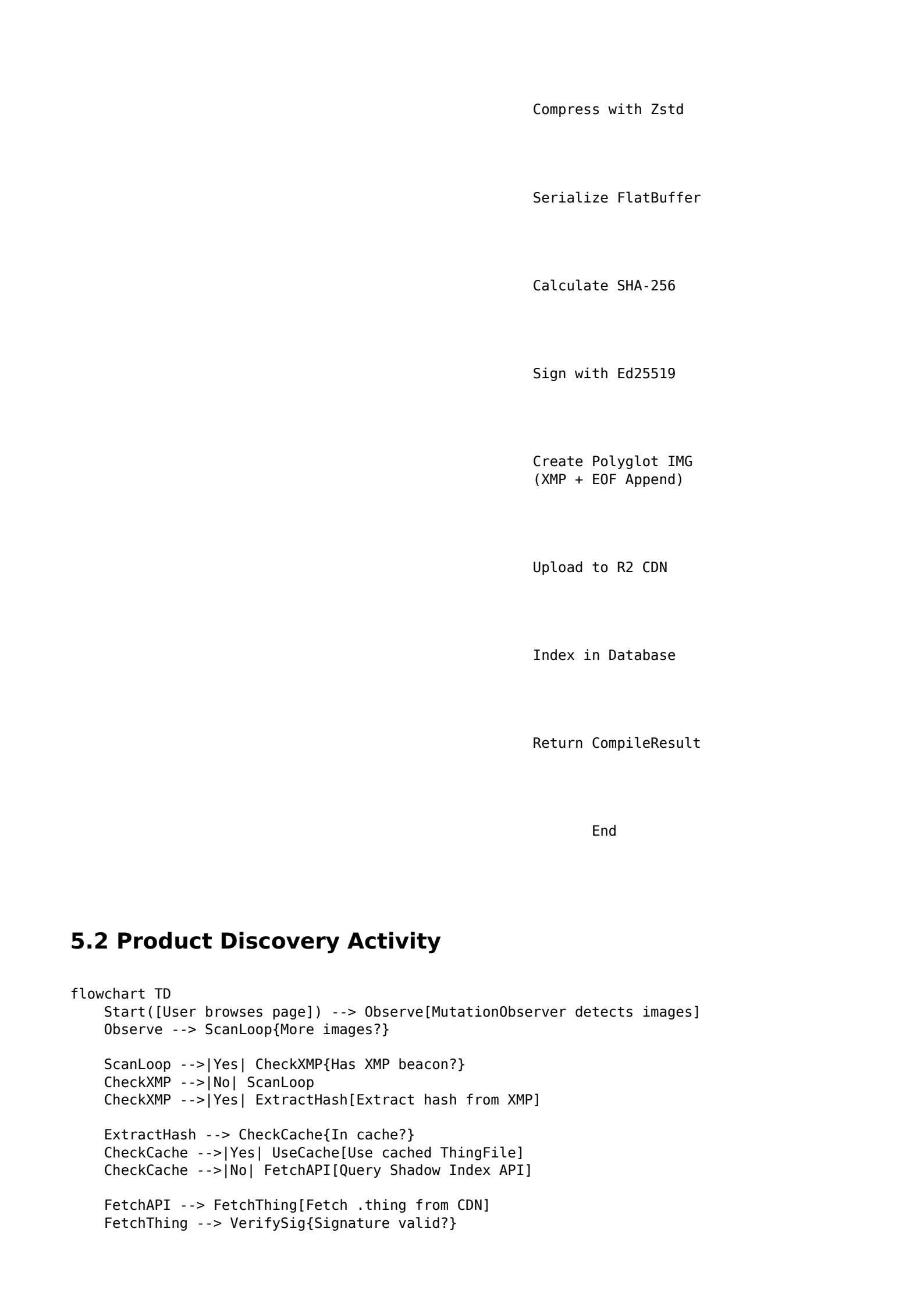
    Sign --> CreateGist[Create Gist Object]
    CreateGist --> InjectXMP[Inject XMP Metadata into Image]
    InjectXMP --> AppendEOF[Append .thing Binary to Image EOF]

    AppendEOF --> UploadThing[Upload .thing to R2]
    UploadThing --> UploadImage[Upload Polyglot Image to R2]
    UploadImage --> IndexHash[Index Hash in Database]

    IndexHash --> GenerateURI[Generate matter:// URI]
    GenerateURI --> Return[Return CompileResult]
    Return --> End4([End])
```

\1





5.2 Product Discovery Activity

```
flowchart TD
    Start([User browses page]) --> Observe[MutationObserver detects images]
    Observe --> ScanLoop{More images?}

    ScanLoop -->|Yes| CheckXMP{Has XMP beacon?}
    CheckXMP -->|No| ScanLoop
    CheckXMP -->|Yes| ExtractHash[Extract hash from XMP]

    ExtractHash --> CheckCache{In cache?}
    CheckCache -->|Yes| UseCache[Use cached ThingFile]
    CheckCache -->|No| FetchAPI[Query Shadow Index API]

    FetchAPI --> FetchThing[Fetch .thing from CDN]
    FetchThing --> VerifySig{Signature valid?}
```

```
    VerifySig -->|No| ShowWarning[Show warning badge]
    ShowWarning --> ScanLoop

    VerifySig -->|Yes| CacheIt[Cache in IndexedDB]
    CacheIt --> UseCache

    UseCache --> RenderOverlay[Render Shadow DOM overlay]
    RenderOverlay --> ScanLoop

    ScanLoop -->|No| WaitMutation[Wait for DOM changes]
    WaitMutation --> Observe
```

6. State Machine Diagrams

6.1 ThingFile Lifecycle States

```
stateDiagram-v2
    [*] --> Draft: Product JSON received

    Draft --> Validating: validate()
    Validating --> Invalid: Validation failed
    Validating --> Valid: Validation passed

    Invalid --> [*]: Discard

    Valid --> Processing: compile()
    Processing --> Vectorizing: Fetch CLIP embedding
    Vectorizing --> Compressing: Apply Zstd
    Compressing --> Serializing: Build FlatBuffer
    Serializing --> Signing: Apply Ed25519

    Signing --> Signed: Signature complete

    Signed --> Embedding: Create polyglot image
    Embedding --> Uploading: Push to CDN

    Uploading --> Published: Available on CDN

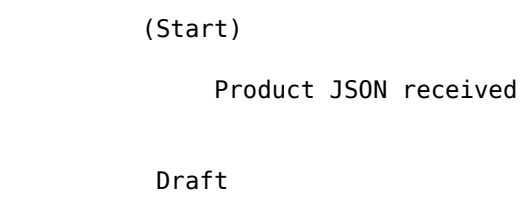
    Published --> Cached: Fetched by client
    Cached --> Verified: Signature validated
    Cached --> Rejected: Signature invalid

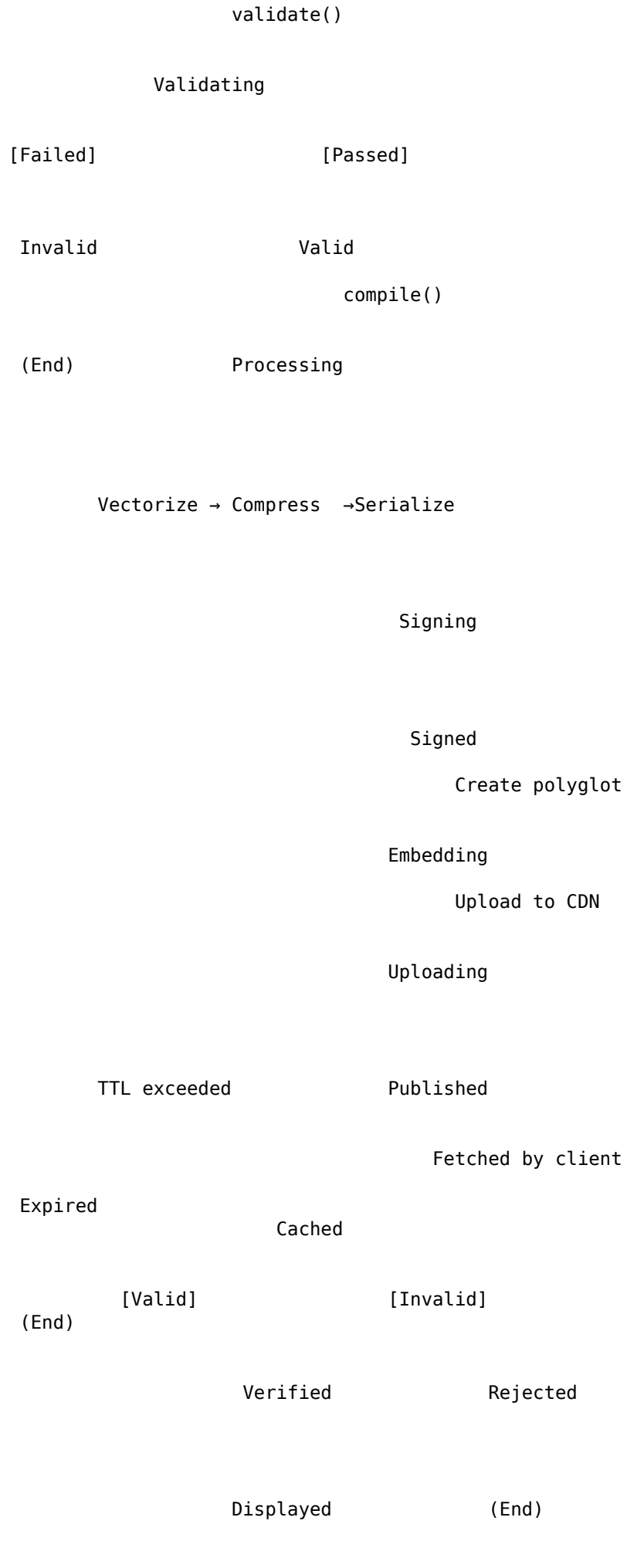
    Verified --> Displayed: Shown to user
    Displayed --> Published: Cache expires

    Rejected --> [*]: Discard

    Published --> Expired: TTL exceeded
    Expired --> [*]: Remove from index
```

\1




validate()
Validating
[Failed]
[Passed]
Invalid
Valid
compile()
(End)
Processing
Vectorize → Compress →Serialize
Signing
Signed
Create polyglot
Embedding
Upload to CDN
Uploading
TTL exceeded
Published
Fetched by client
Expired
Cached
[Valid]
[Invalid]
(End)
Verified
Rejected
Displayed
(End)

6.2 Browser Extension States

```
stateDiagram-v2
    [*] --> Idle: Extension loaded

    Idle --> Scanning: Page loaded / DOM changed
    Scanning --> Idle: No beacons found

    Scanning --> BeaconDetected: XMP beacon found
    BeaconDetected --> CheckingCache: Extract hash

    CheckingCache --> CacheHit: Found in IndexedDB
    CheckingCache --> CacheMiss: Not in cache

    CacheMiss --> Fetching: Request from API
    Fetching --> Verifying: .thing received
    Fetching --> Error: Fetch failed

    Verifying --> Verified: Signature valid
    Verifying --> Rejected: Signature invalid

    Verified --> Caching: Store in IndexedDB
    Caching --> CacheHit

    CacheHit --> Rendering: Build overlay
    Rendering --> Displayed: Show to user

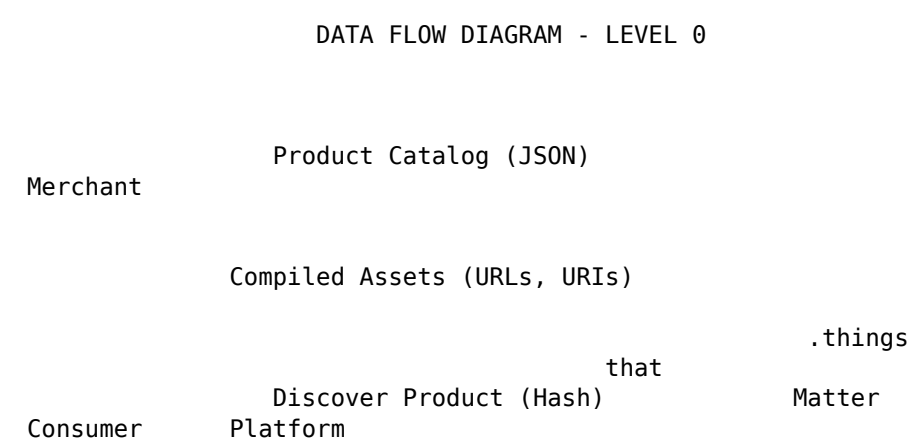
    Displayed --> Idle: User scrolls away
    Displayed --> Purchasing: User clicks Buy

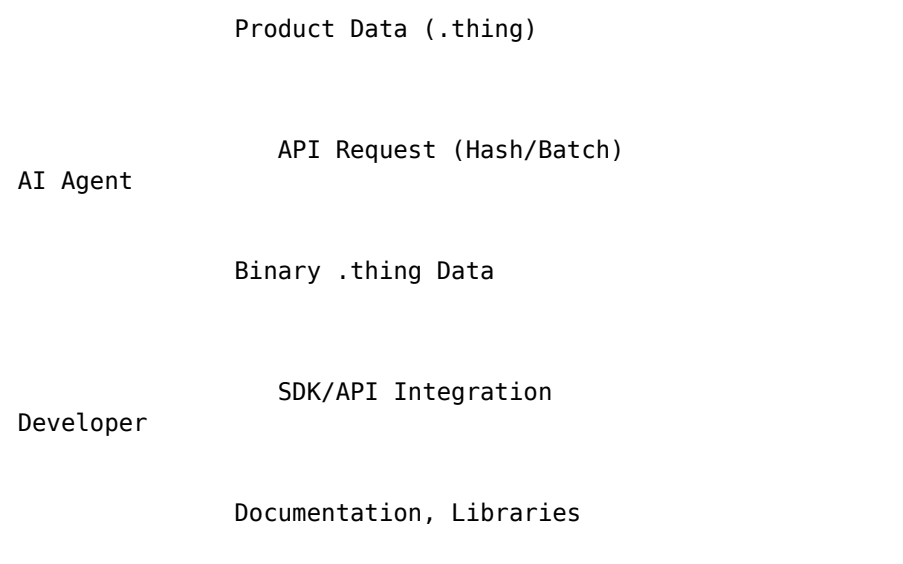
    Purchasing --> Redirecting: Open checkout URL
    Redirecting --> Idle: Navigation complete

    Error --> Idle: Retry later
    Rejected --> Idle: Show warning
```

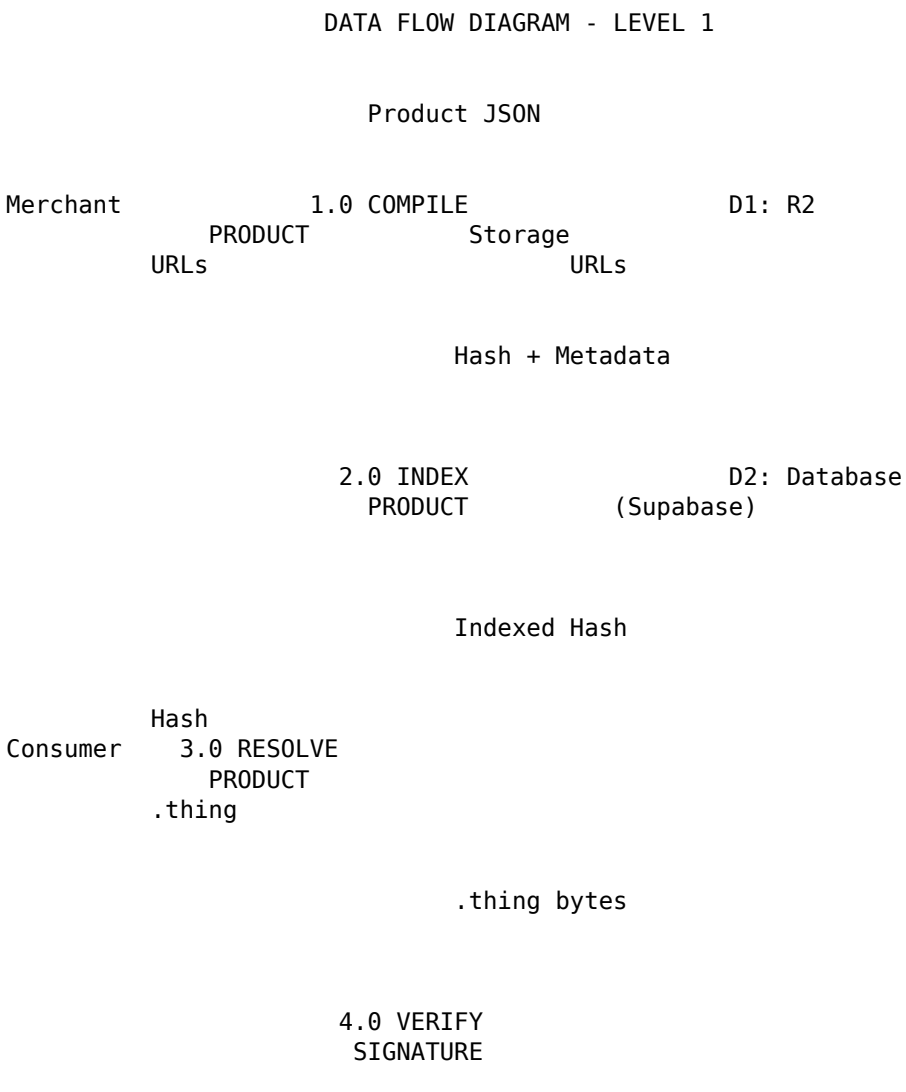
7. Data Flow Diagrams

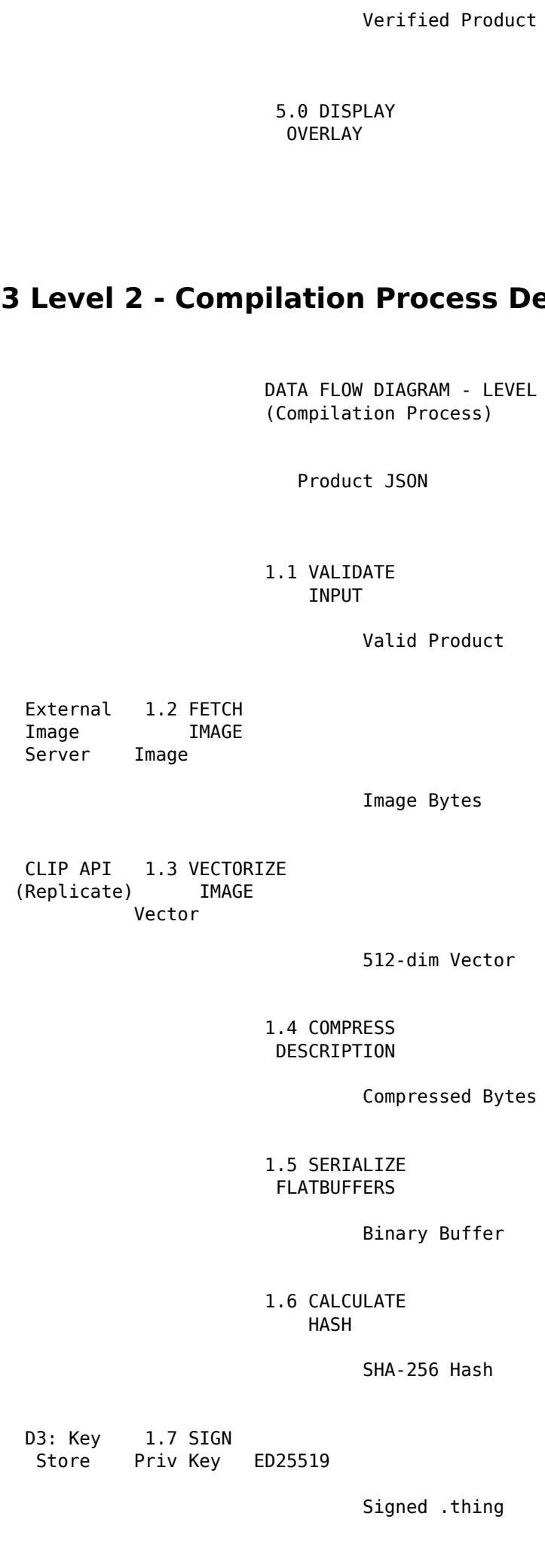
7.1 Level 0 - Context Diagram



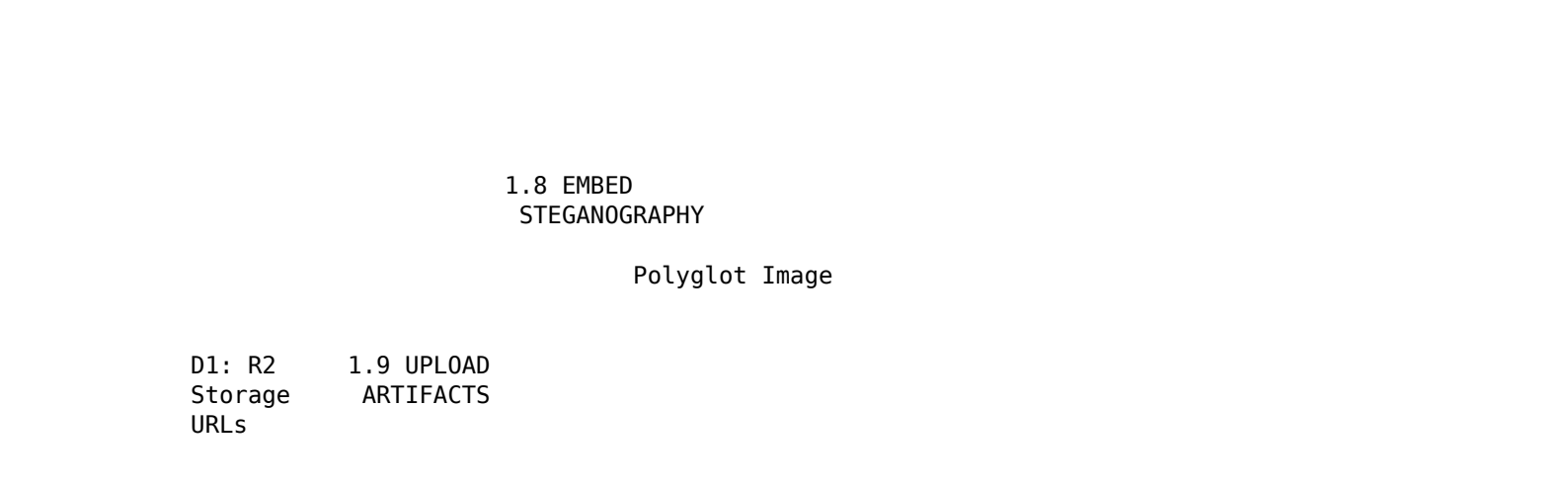


7.2 Level 1 - System Processes





7.3 Level 2 - Compilation Process Detail



8. Entity Relationship Diagram

8.1 Database Schema ERD

```
erDiagram
    MERCHANT ||--o{ PRODUCT : owns
    MERCHANT ||--o{ API_KEY : has
    PRODUCT ||--|| THING_FILE : compiles_to
    THING_FILE ||--|| POLYGLOT_IMAGE : embeds_in
    THING_FILE ||--o{ RESOLUTION_LOG : tracked_by
    PRODUCT ||--o{ PRODUCT_TAG : has
    TAG ||--o{ PRODUCT_TAG : applies_to

    MERCHANT {
        uuid id PK
        string name
        string email
        string api_key_hash
        timestamp created_at
        timestamp updated_at
        boolean active
    }

    API_KEY {
        uuid id PK
        uuid merchant_id FK
        string key_hash
        string name
        timestamp created_at
        timestamp expires_at
        boolean active
    }

    PRODUCT {
        uuid id PK
        uuid merchant_id FK
        string external_id
        string title
        text description
        bigint price_cents
        string currency
        string sku
        string image_url
        string checkout_url
        string inventory_status
        timestamp created_at
        timestamp updated_at
    }
```

```
    THING_FILE {
        string hash PK
        uuid product_id FK
        string thing_url
        string image_url
        string matter_uri
        bytea clip_vector
        bytea signature
        bytea signer_pubkey
        timestamp created_at
        timestamp expires_at
        integer version
    }

    POLYGLOT_IMAGE {
        string hash PK
        string thing_hash FK
        string cdn_url
        string mime_type
        integer file_size
        boolean xmp_intact
        boolean eof_intact
        timestamp created_at
    }

    RESOLUTION_LOG {
        uuid id PK
        string thing_hash FK
        string client_ip
        string user_agent
        string referer
        timestamp resolved_at
    }

    TAG {
        uuid id PK
        string name
        string category
    }

    PRODUCT_TAG {
        uuid product_id FK
        uuid tag_id FK
    }
```

\1

```
                    ENTITY RELATIONSHIP DIAGRAM


      MERCHANT                        API_KEY

 PK id: uuid                  PK id: uuid
    name: string                 FK merchant_id: uuid
    email: string                   key_hash: string
    api_key_hash                    name: string
    created_at                      created_at
    updated_at                      expires_at
    active: boolean                 active: boolean


           owns
```



9. Deployment Diagram

9.1 Production Infrastructure

```
graph TB
    subgraph "Client Layer"
        Browser[Web Browser]
        Extension[Lens Extension]
        Agent[AI Agent]
    end

    subgraph "CDN Layer"
        CF[Cloudflare CDN]
    end

    subgraph "Application Layer - Vercel"
        Dashboard[Merchant Dashboard<br/>Next.js]
```

```
        API[Shadow Index API<br/>Edge Functions]
    end

    subgraph "Compute Layer"
        Compiler[Commerce Compiler<br/>Serverless Function]
    end

    subgraph "Data Layer"
        subgraph "Supabase"
            PG[(PostgreSQL)]
            Auth[Auth Service]
        end
        Redis[(Redis Cloud)]
        R2[(Cloudflare R2)]
    end

    subgraph "External Services"
        CLIP[Replicate CLIP API]
        ChromeStore[Chrome Web Store]
    end

    Browser --> CF
    Extension --> CF
    Agent --> CF

    CF --> Dashboard
    CF --> API
    CF --> R2

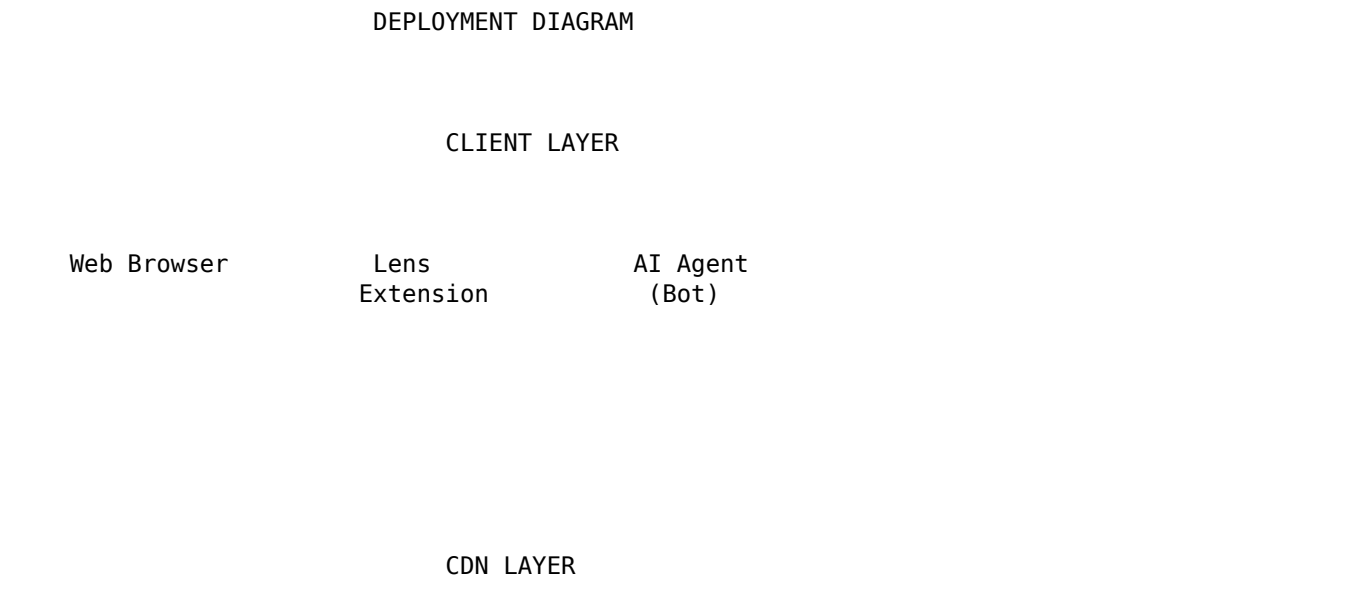
    Dashboard --> API
    API --> Compiler
    API --> PG
    API --> Redis

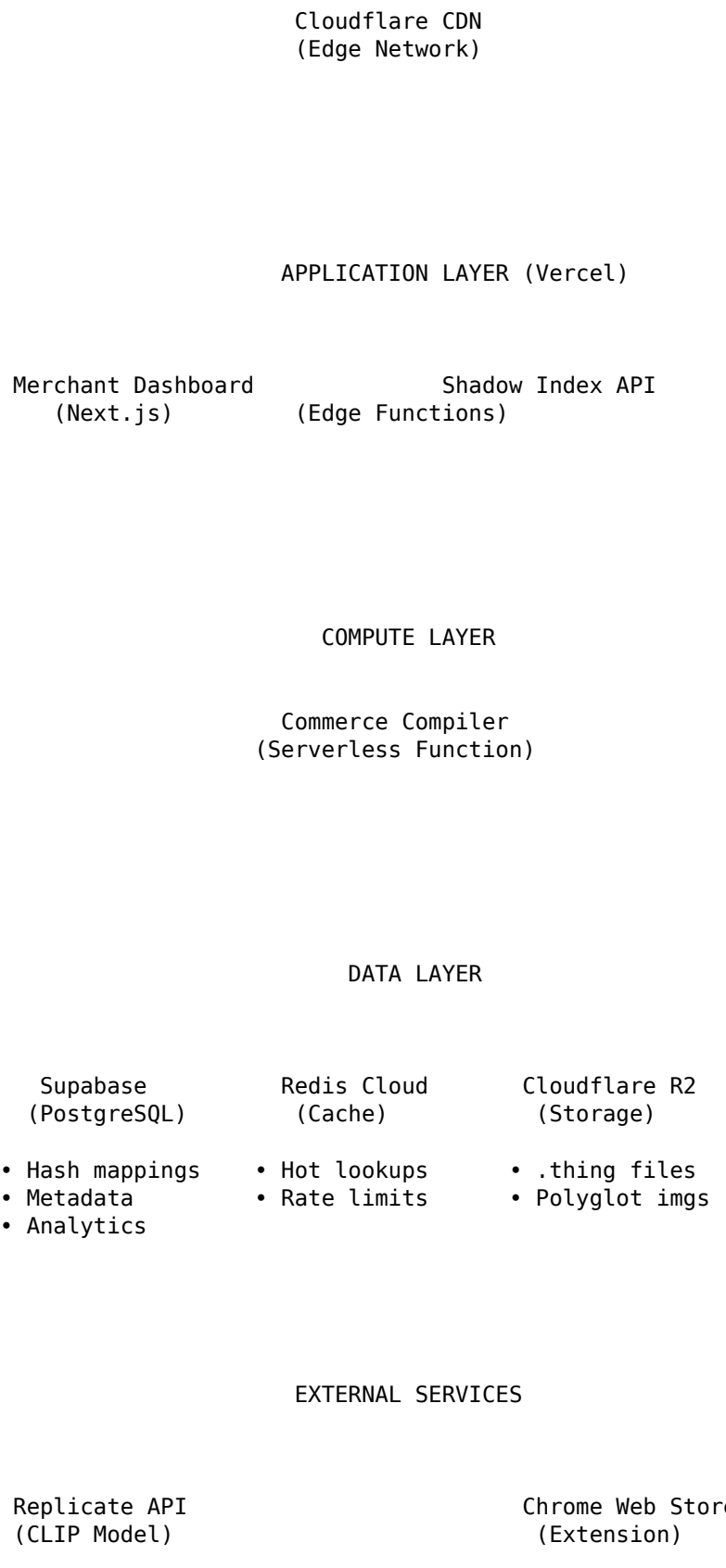
    Compiler --> CLIP
    Compiler --> R2

    Dashboard --> Auth

    Extension -.-> ChromeStore
```

\1





10. Component Diagrams

10.1 Commerce Compiler Components

```
graph TB
    subgraph "Commerce Compiler (matter-core)"
        subgraph "Ingestion Layer"
            Validator[Input Validator<br/>Zod/Pydantic]
            ImageFetcher[Image Fetcher<br/>HTTP Client]
        end

        subgraph "Processing Layer"
            CLIPClient[CLIP Client<br/>Replicate API]
            Compressor[Compressor<br/>Zstd]
            Serializer[Serializer<br/>FlatBuffers]
            Hasher[Hasher<br/>SHA-256]
            Signer[Signer<br/>Ed25519]
        end

        subgraph "Emission Layer"
            StegEngine[Steganography Engine]
            XMPInjector[XMP Injector]
            EOFAppender[EOF Appender]
            URIGenerator[URI Generator<br/>matter://]
        end

        subgraph "Output Layer"
            Uploader[CDN Uploader<br/>R2 Client]
            Indexer[Index Writer<br/>DB Client]
        end
    end

    Validator --> ImageFetcher
    ImageFetcher --> CLIPClient
    CLIPClient --> Compressor
    Compressor --> Serializer
    Serializer --> Hasher
    Hasher --> Signer
    Signer --> StegEngine
    StegEngine --> XMPInjector
    XMPInjector --> EOFAppender
    EOFAppender --> URIGenerator
    URIGenerator --> Uploader
    Uploader --> Indexer
```

10.2 Browser Extension Components

```
graph TB
    subgraph "Browser Extension (Lens)"
        subgraph "Content Script"
            DOMObserver[DOM Observer<br/>MutationObserver]
            XMPDetector[XMP Detector<br/>Image Scanner]
            URIHandler[URI Handler<br/>matter:// Protocol]
            OverlayManager[Overlay Manager<br/>Shadow DOM]
        end

        subgraph "Background Worker"
            APIClient[API Client<br/>Fetch]
            WASMVerifier[WASM Verifier<br/>libmatter.wasm]
            CacheManager[Cache Manager<br/>IndexedDB]
            StateManager[State Manager]
        end

        subgraph "UI Components"
            ProductCard[Product Card]
            SettingsPage[Settings Page]
```

```
            WarningBadge[Warning Badge]
        end
    end

    DOMObserver --> XMPDetector
    XMPDetector --> StateManager
    URIHandler --> StateManager
    StateManager --> CacheManager
    StateManager --> APIClient
    APIClient --> WASMVerifier
    WASMVerifier --> CacheManager
    CacheManager --> OverlayManager
    OverlayManager --> ProductCard
    OverlayManager --> WarningBadge
```

10.3 Shadow Index API Components

```
graph TB
    subgraph "Shadow Index API"
        subgraph "API Routes"
            CompileRoute[POST /compile]
            BatchRoute[POST /batch]
            ResolveRoute[GET /resolve/:hash]
            GistRoute[GET /gist/:hash]
            HealthRoute[GET /health]
        end

        subgraph "Middleware"
            AuthMiddleware[Authentication<br/>API Key Validation]
            RateLimiter[Rate Limiter<br/>Token Bucket]
            Validator[Request Validator<br/>Zod Schema]
            ErrorHandler[Error Handler]
        end

        subgraph "Services"
            CompilerService[Compiler Service]
            ResolverService[Resolver Service]
            CacheService[Cache Service<br/>Redis]
            StorageService[Storage Service<br/>R2]
        end

        subgraph "Data Access"
            DBClient[Database Client<br/>Supabase]
            RedisClient[Redis Client]
            R2Client[R2 Client]
        end
    end

    CompileRoute --> AuthMiddleware
    BatchRoute --> AuthMiddleware
    ResolveRoute --> RateLimiter
    GistRoute --> RateLimiter

    AuthMiddleware --> Validator
    RateLimiter --> Validator
    Validator --> ErrorHandler

    ErrorHandler --> CompilerService
    ErrorHandler --> ResolverService

    CompilerService --> StorageService
    CompilerService --> DBClient
```

```
    ResolverService --> CacheService
    CacheService --> RedisClient
    StorageService --> R2Client
```

11. Appendix: Diagram Legend

Symbol Reference

```
| Symbol | Meaning |
|--------|---------|
|  | System/Container |
|  | Actor/User |
|  | Decision Point |
|  | Data Flow |
|  | Control Flow |
|  | Composition |
|  | Aggregation |
|  | Dependency |
|   | Optional/Async |
```

Color Coding (for rendered Mermaid)

```
| Color | Meaning |
|-------|---------|
| Blue | Core System Components |
| Green | External Services |
| Orange | Data Stores |
| Purple | User Interfaces |
| Red | Security Components |
```

12. Document Control

```
| Version | Date | Author | Changes |
|---------|------|--------|---------|
| 1.0 | Jan 21, 2026 | M. Mboweni | Initial comprehensive diagrams |
```

\1 1.0
\1 January 21, 2026
\1 01-PROJECT-CHARTER, 02-TECHNICAL-ARCHITECTURE, 03-PRD